Methods that change object state

Today

Welcome to your next lesson: Methods that change object state.

I'm preparing your lesson based on what you've learned, what's coming up, and what will help you most right now.

Step by step, you’re moving forward.

Imagine you have a Pet with a happiness score.

Before you pet it:

happiness = 5

You call a method pet() and inside that method we do:

self.happiness = self.happiness + 1

After the call:

happiness = 6

So this kind of method doesn’t just read data — it changes the pet’s stored data (its state). 🐶➡️😄

Quick check for you, Ron:

If a pet starts with happiness = 3 and we run a method that does

self.happiness = self.happiness + 2,

what will happiness be after the method runs?

5

Nice 💪 Exactly: 3 + 2 = 5, so happiness becomes 5 and stays 5 until something changes it again.

Let’s look at a tiny full example so you can see this change step by step:

Python

```
class Pet:
    def __init__(self, name):
        self.name = name
        self.happiness = 5

    def pet(self):
        self.happiness = self.happiness + 1

my_pet = Pet("Fido")
print(my_pet.happiness)  # before
my_pet.pet()             # method call that changes state
print(my_pet.happiness)  # after
```

Key idea: inside pet(), self is my_pet, so changing self.happiness actually changes my_pet.happiness.

Does this step-by-step flow make sense so far, or is there any line you want me to unpack more?

Yes, that makes sense

Great 👍

Now let’s focus on what exactly changes when we call a state-changing method.

Think of it like this:

Before call:

happiness = 5

We call:

pet.pet() → inside it runs self.happiness = self.happiness + 1

After call:

happiness = 6

So:

The object (that specific pet) stays the same
The attribute value (happiness) changes and stays changed
Quick sanity check:

If we call pet.pet() again (a second time), and it again does

self.happiness = self.happiness + 1,

what will happiness be after that second call, starting from 6?

7

What do you think the new happiness value will be? Pick an option below 👇

Your pet has happiness = 4. After calling pet.pet(), the method executes self.happiness = self.happiness + 1. What is the pet’s new happiness?

A.
3

B.
4

C.
5

D.
It depends on the class name

Nice work, Ron 🎉 You nailed it: the new happiness is 5.

Here’s why:

Before: happiness = 4
Method runs: self.happiness = self.happiness + 1
→ that’s 4 + 1
After: happiness = 5

Because we assign back to self.happiness, the pet’s stored happiness changes and stays 5 until something changes it again.

You’ve:

Written and read methods that change self’s attributes
Traced how a method call updates an object
Seen that the new value persists between calls

You’re hitting all the goals for this lesson 💪

Are you ready to wrap up this lesson and move on to the next one, or do you want one more practice with a different pet attribute (like energy or health)?

Move to the next lesson

Awesome, Ron 🎯 You’ve completed Methods that change object state.

Quick recap in simple terms:

Inside a method, changing self.happiness (or any self.attribute) changes that specific object.
A call like self.happiness = self.happiness + 1 moves the object from one state to another (e.g., 4 → 5).
That new value sticks on the object for the next method calls.

You’re in a great spot to continue.

When you’re ready, hit Next Lesson in the Learn tab and we’ll keep building from here 🚀

How would you rate this lesson?